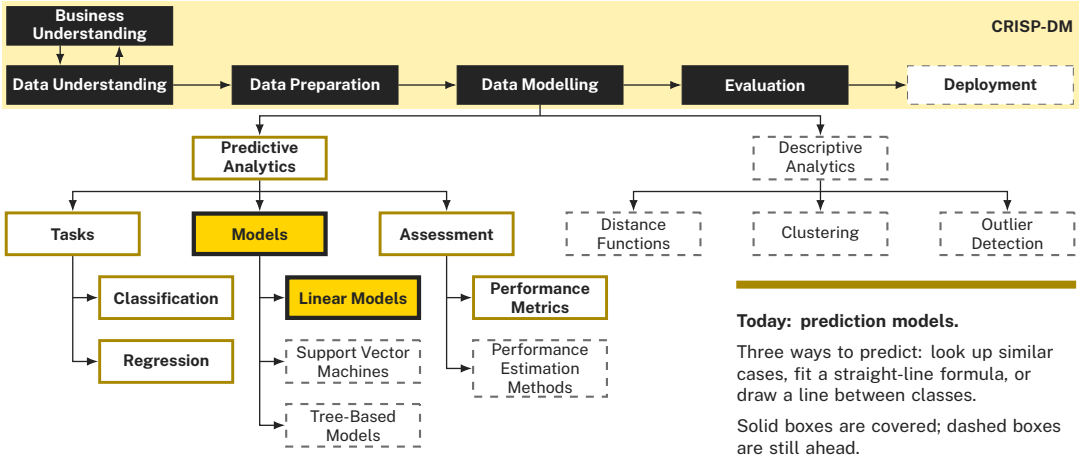
DALHOUSIE UNIVERSITY

Prediction Models

Nearest neighbours, linear regression, and linear discriminants

Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Where prediction models sit

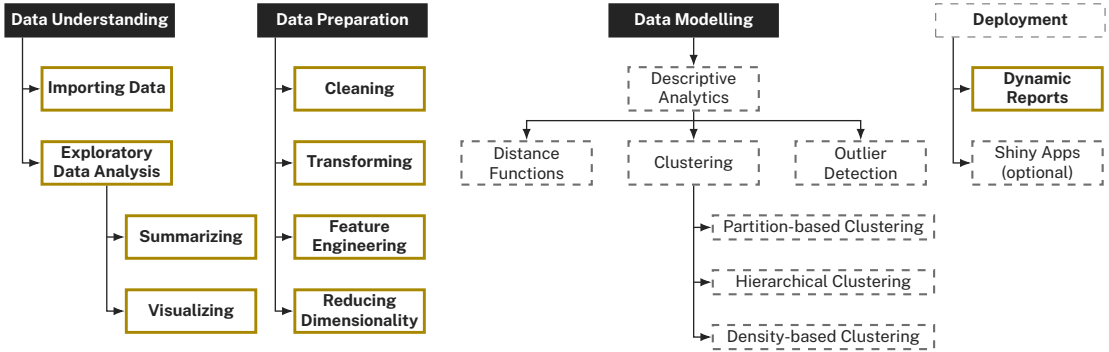


Today: prediction models.

Three ways to predict: look up similar cases, fit a straight-line formula, or draw a line between classes.

Solid boxes are covered; dashed boxes are still ahead.

The rest of the map, for orientation



Everything to the left of Data Modelling has already been covered. Solid boxes are covered, dashed boxes are still ahead, apart from Shiny Apps, which stays optional.

K nearest neighbours

DALHOUSIE UNIVERSITY

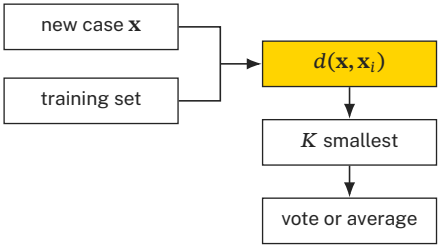
The first model of the course keeps the training set and answers each new question by looking it up.

K nearest neighbours (KNN) I

Given a new test case $\mathbf{x}$:

1. Search in the training set for the K most similar cases to the new case.
2. Use the target values of these cases to obtain a prediction. In classification, predict the majority class; in regression, predict the average.

How do we measure similarity? We can use **Euclidean distance** over the space spanned by the p predictors.



$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{(x_1 - x_{i1})^2 + \cdots + (x_p - x_{ip})^2}$$

nothing is fitted in advance, since every prediction re-runs the whole search

Three neighbours, by hand

Five training cases, two attributes, a nominal target and a numeric target. The query is $\mathbf{x} = (5.0, 3.0)$.

Case	x_1	x_2	class	y	$d(\mathbf{x}, \mathbf{x}_i)$
A	4.6	3.2	YES	12	$\sqrt{0.16 + 0.04} = 0.447$
B	5.4	3.4	YES	14	$\sqrt{0.16 + 0.16} = 0.566$
C	5.1	2.3	NO	22	$\sqrt{0.01 + 0.49} = 0.707$
D	6.3	3.3	NO	25	$\sqrt{1.69 + 0.09} = 1.334$
E	4.4	1.9	NO	30	$\sqrt{0.36 + 1.21} = 1.253$

The three smallest distances are A, B, C.

Classification, $K = 3$
Votes: YES, YES, NO ⇒ predict YES.

Regression, $K = 3$

$$\hat{y} = \frac{12 + 14 + 22}{3} = 16.0$$

Change K and the answer changes
$K = 1$ gives YES and $\hat{y} = 12$.
$K = 5$ gives NO (3 votes to 2) and $\hat{y} = 20.6$.

Same data, same distance, three different predictions. K is not a detail.

Why the features must be scaled first

A Euclidean distance adds squared differences, so a variable with a wide numeric range dominates the sum no matter how relevant it is.

RAW UNITS

Case	income ($)	age (yr)	d to $\mathbf{x}$
$\mathbf{x}$	52 000	30	—
P	52 300	61	301.6
Q	55 000	31	3000.0

The nearest neighbour is P, a 61-year-old. Income alone gives a distance of 300; the 31-year age gap barely moves it, to 301.6.

Same data, after standardizing

DALHOUSIE UNIVERSITY

STANDARDIZED

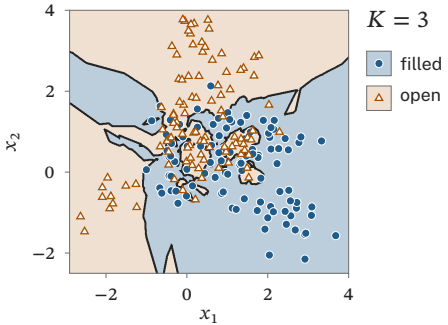
Case	income z	age z	d to $\mathbf{x}$
$\mathbf{x}$	−0.20	−0.85	—
P	−0.18	1.22	2.07
Q	0.00	−0.78	0.21

The nearest neighbour is now Q, the 31-year-old. Nothing about the data changed, only the units.

Neither library scales for you: `class::knn` and `KNeighborsClassifier` take the columns as given. Standardize on the *training* statistics, then apply the same centre and scale to the test set.

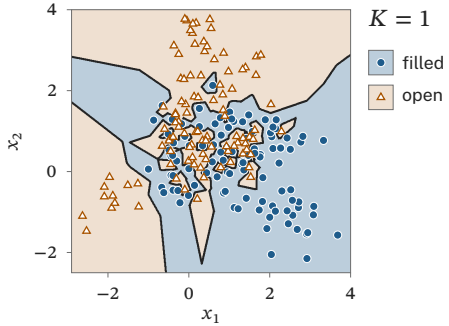
K nearest neighbours (KNN) II

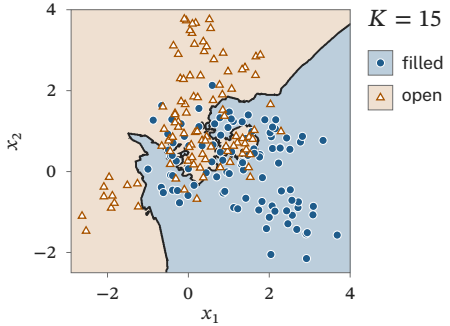
- No assumptions about the functional form of the mapping.
- A "**lazy**" learner.
  - "Learning" the model just means storing the training set.
- Quick and easy training.
- Prediction more computationally expensive.
- K is what is called a **hyperparameter**.
  - Hyperparameters control the learning process; their values are not learned from the data.



Simulated data, 200 cases with two numeric predictors, x_1 and x_2, and two classes of 100. Orange predicts open triangles; blue predicts filled circles. After Hastie, Tibshirani and Friedman (2009), Section 2.3.

Small K, large K: the same data twice



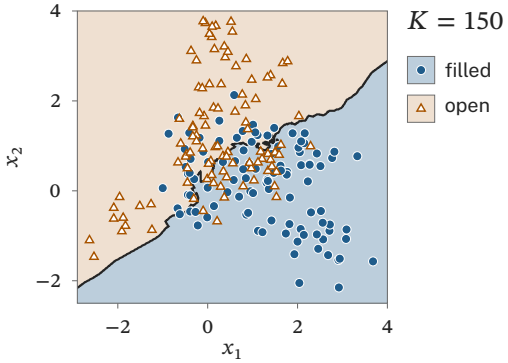


Small K: captures fine-grained patterns, but may **overfit**. Every island is one case having its say alone.

Large K: more stable, but may **underfit**. Averaging the same cases fifteen at a time gives one smooth frontier.

Identical 200 training cases, Euclidean distance and code; only K changed. All coordinates are displayed.

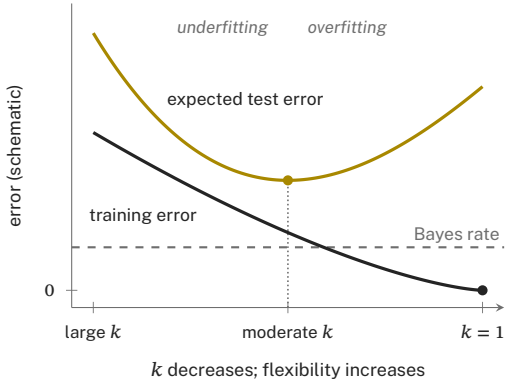
How large is too large



- **Starting scale:** try $k \approx \sqrt{n}$, then select k by validation. Lower-case k and K name the same hyperparameter.
  - Here $n = 200$, so $\sqrt{n} \approx 14$, a starting scale, not proof that $K = 15$ is optimal.
- **Global estimation:** search for the ideal k for a given data set.
- **Local estimation:** try to estimate the ideal k for each test case.

At $K = 150$ out of 200 the neighbourhood is most of the training set, the model has stopped being local.

Reading the error curves: a schematic



- At $K = 1$, training error is **zero** when cases include themselves and identical predictors have consistent labels.
- Expected test error may fall, then rise, as flexibility grows.
- The **Bayes rate** bounds expected error on fresh cases. Training scores and finite test estimates may fall below it.
- This is a qualitative pattern, without measured error rates.

Choose K by validation; assess once on untouched test cases.

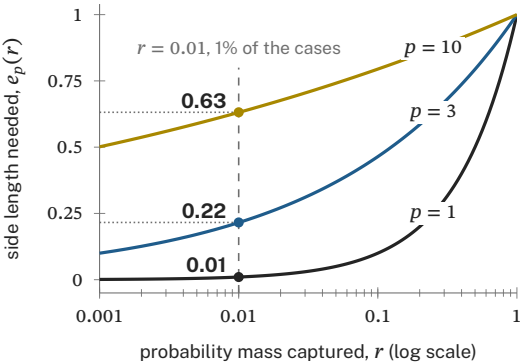
The curse of dimensionality

"Nearest" has to mean something. How wide must a neighbourhood be to hold 1% of evenly spread cases?

- 1 predictor: 1% of each axis.
- 3 predictors: 22% of each axis.
- 10 predictors: 63% of each axis.

Width $= r^{1/p}$ for a share r in p dimensions. At ten predictors the "neighbours" are far away.

What to do. Reduce dimensionality first or select features, or weight the distance.



At $r = 0.01$ and $p = 10$ the side length is already 0.63.

kNN in R and Python

The data. `iris`, 150 flowers of three species, 50 each, measured in cm on four traits. Flower 134 is the new case.

R

```r
library(class)
tr <- iris[-134, 1:4]
te <- iris[134, 1:4]
cl <- iris$Species[-134]
set.seed(3141)
knn(tr, te, cl, k = 4)   # ties: random
m <- colMeans(tr); s <- apply(tr, 2, sd)
knn(scale(tr, m, s), scale(te, m, s),
    cl, k = 4)
```

Python

```python
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
X, y = load_iris(return_X_y=True, as_frame=True)
tr, te, cl = X.drop(133), X.iloc[[133]], y.drop(133)
KNeighborsClassifier(4).fit(tr, cl).predict(te)
pipe = make_pipeline(StandardScaler(),
                     KNeighborsClassifier(4))
pipe.fit(tr, cl).predict(te)  # scaler sees tr only
```

Flower 134 ties 2 to 2 at $k = 4$. R breaks ties at random, hence the seed; scikit-learn takes the lowest class index. **Neither scales for you.**

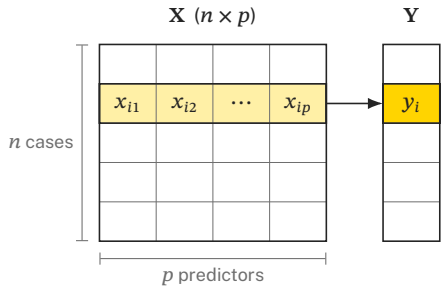
Multiple linear regression

DALHOUSIE UNIVERSITY

The opposite bargain, assume the shape of the function, and buy back interpretation and speed.

The Data as a Table

- Multiple linear regression is probably the most used statistical method. It predicts a **numeric** target from several predictors.
- The training data is a table. Each of the n rows is one case; the p predictor columns form $\mathbf{X}$ and the target column is $\mathbf{Y}$.
- The goal is a formula that turns a row of predictor values into a good guess of its target, an approximation of the unknown $f()$.



$\mathbf{Y}$ is numeric here; a nominal target would make it classification

Three Decisions Behind Every Fit

$$\text{target} = \text{model}(\text{predictors}) + \text{error}$$

The model form

The shape is a *choice*. Fix it, a weighted sum here, and the only unknowns left are the weights β.

The error term

The error ϵ absorbs everything the model does not explain, noise, missing predictors, a wrong shape. Leaving it out would claim the model is exact.

The criterion

"Best" weights depend on how misses are charged. Squared error, absolute error and likelihood give three different answers from one data set.

Blaming the data for a poor fit is usually a complaint about the first decision.

A Weighted Sum, Fitted by Squared Misses

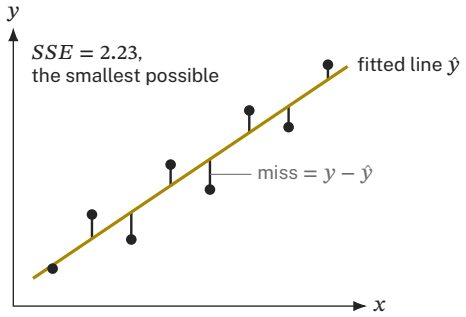
The model is a weighted sum,

$$\hat{y} = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p,$$

and keeps the weights with the smallest **sum of squared errors**,

$$SSE = \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2.$$

Squaring keeps misses positive, has an exact answer, and suits Normal errors, but one outlier can move the line.



Handling Categorical Predictors

A weighted sum needs numbers, so a category becomes **dummy columns**.

1. A variable with C levels gets $C - 1$ columns.
2. Each column is 1 for its level and 0 otherwise.

`lm` does this automatically. In Python, use `pd.get_dummies(..., drop_first=True)`; `OneHotEncoder` keeps all C columns unless told `drop="first"`.

size	sizemedium	sizesmall
large	0	0
medium	1	0
small	0	1

Three levels, two columns. `large` is the baseline, the row of two zeros. A third column would repeat information and leave `lm` without a unique answer.

How `lm()` Finds the Weights

DALHOUSIE UNIVERSITY

Solved, not searched

Least squares has an exact answer, so there is no trial and error and no tuning. `lm` computes it with a QR decomposition, and `LinearRegression` with an SVD routine. The work grows with the number of rows times the square of the number of predictors.

It needs distinct columns

The answer is unique only if no column can be rebuilt from the others. Duplicate a column, or keep all C dummies of a factor next to an intercept, and there is no single best set of weights; `lm` reports `NA`. Columns that are *nearly* copies give unstable weights.

What Linear Regression Buys and Costs

DALHOUSIE UNIVERSITY

What it buys

- One weight per predictor, each an additive effect you can read.
- Fast, exact and well studied.
- Works well when the relationship is close to a straight line.

What it costs

- It assumes the shape of the relationship. When the truth is curved, the model is wrong, not merely noisy.

kNN makes the opposite bargain, no assumed shape, paid for in query cost, sensitivity to scale, and nothing to report but a prediction.

The Algae Data

- **200 water samples** from European rivers, collected over about a year (UCI COIL 1999 data).
- **The target, `a1`**, is how frequent one harmful algae species is in the sample, from 0 to 89.8 with a median of 7.0.
- **Eleven predictors**, three categories describing the river and eight chemical measurements.
- **140 more samples**, with their `a1` kept aside, test the model at the end.
- 16 samples have gaps; two are too incomplete to use.

`season`	autumn, spring, summer, winter
`size`	river size, small to large
`speed`	water speed, low to high
`mxPH`	maximum pH
`mnO2`	minimum oxygen
`Cl`	chloride
`NO3`	nitrates
`NH4`	ammonium
`oPO4`	orthophosphate
`PO4`	total phosphate
`Chla`	chlorophyll

Fitting the Algae Model

R

```r
source("mixed_knn_impute.R")
algae <- read.csv("data/algae.csv")
algae <- algae[-c(62, 199), ]  # 2 incomplete
imputed <- mixed_knn_impute(algae[, 1:11])
clean.algae <- cbind(imputed, a1 = algae$a1)
la1 <- lm(a1 ~ ., clean.algae[, 1:12])
summary(la1)
```

Python

```python
import pandas as pd, statsmodels.formula.api as smf
from sklearn.impute import KNNImputer
algae = pd.read_csv("data/algae.csv")
algae = algae.drop(index=[61, 198])  # 0-based
num = algae.columns[3:11]  # numeric predictors
raw_numeric = algae[num].copy()
imp = KNNImputer(n_neighbors=10)  # numeric only
algae[num] = imp.fit_transform(raw_numeric)
rhs = " + ".join(algae.columns[:11])  # no "~ ."
la1 = smf.ols(f"a1 ~ {rhs}", algae).fit()
la1.summary()
```

Three decisions come before the fit. Drop the two incomplete samples, fill the remaining gaps by kNN imputation, and use every other column as a predictor.

Rows 62 and 199 in R are 61 and 198 in pandas. Python's simpler imputer gives slightly different numbers.

Reading the Coefficients

(Intercept)	42.873675	mxPH	−3.545649
seasonspring	3.628157	mnO2	1.024855
seasonsummer	0.707369	Cl	−0.040249
seasonwinter	3.605094	NO3	−1.504920
sizemedium	3.317693	NH4	0.001621
sizesmall	9.749954	oPO4	−0.004792
speedlow	3.818859	PO4	−0.052880
speedmedium	0.229648	Chla	−0.085500

- Each number is how much the prediction of `a1` moves per unit of its predictor, with the others held fixed.
- `season`, `size` and `speed` are categories. `lm` turns each into dummy columns and leaves one level out as the baseline: `autumn`, `large` and `high`.
- A dummy's number compares its level with the baseline. `sizesmall` = 9.75 means small rivers are predicted 9.75 higher than large ones.

Which Predictors Matter

	Estimate	Std. Error	t value	
(Intercept)	42.873675	24.157169	1.775	.
seasonspring	3.628157	4.140971	0.876	
seasonsummer	0.707369	4.023217	0.176	
seasonwinter	3.605094	3.865707	0.933	
sizemedium	3.317693	3.802592	0.872	
sizesmall	9.749954	4.186566	2.329	*
speedlow	3.818859	4.715964	0.810	
speedmedium	0.229648	3.244008	0.071	
mxPH	−3.545649	2.715984	−1.305	
mnO2	1.024855	0.705522	1.453	
Cl	−0.040249	0.033723	−1.194	
NO3	−1.504920	0.551871	−2.727	**
NH4	0.001621	0.001004	1.615	
oPO4	−0.004792	0.039887	−0.120	
PO4	−0.052880	0.030760	−1.719	.
Chla	−0.085500	0.080332	−1.064	

- The Std. Error column holds each weight's SE, roughly

$$\mathrm{SE} \approx \frac{\hat{\sigma}}{s_{x_j}\sqrt{n}},$$

  a mean's $\sigma/\sqrt{n}$ divided by the predictor's spread s_{x_j}; correlated predictors make it larger.
- t = estimate/SE. Above about 2 in size, the weight is clearly away from zero; asterisks in the last column flag it, a dot a near miss.
- Only `sizesmall` and `NO3` pass; one or two of 16 can by chance.

How Good Is the Whole Model

DALHOUSIE UNIVERSITY

Residuals, $y - \hat{y}$

Min	1Q	Median	3Q	Max
−37.625	−11.886	−2.688	7.032	62.278

Fit

Residual standard error	17.65 on 182 df
Multiple R^2	0.3725

- **A typical miss is 17.65** units of `a1`, on a target that runs from 0 to about 90. Not precise.
- $R^2 = 0.37$, so the model explains 37% of the variation in `a1`; 63% is left over.
- R^2 can only rise when a predictor is added, so it cannot tell whether sixteen weights are too many.
- The median miss is below zero and the largest is far out, so the misses are skewed, not bell-shaped.

Simplifying with `step()`

R

```r
final.la1 <- step(la1)  # backward, by AIC
summary(final.la1)
```

`step()` drops the predictor whose removal improves AIC most, a measure of fit that charges for every weight, and stops when no removal helps.

statsmodels has no `step()`, so Python writes the same search out.

Python

```python
import numpy as np
def aic(terms):       # R's extractAIC.lm, k = 2
    rhs = " + ".join(terms) or "1"
    r = smf.ols("a1 ~ " + rhs, algae).fit()
    return (r.nobs*np.log(r.ssr/r.nobs)
            + 2*(r.df_model+1))
keep = list(algae.columns[:11])
while keep:     # step() may walk to the intercept
    d = {t: aic([x for x in keep if x != t])
         for t in keep}
    t = min(d, key=d.get)
    if d[t] >= aic(keep): break
    keep.remove(t)
final = smf.ols("a1 ~ " + (" + ".join(keep) or "1"),
                algae).fit()
```

The Smaller Model Is the Better One

	Estimate	Std. Error	t	
(Intercept)	57.19004	20.96298	2.728	**
sizemedium	2.81275	3.40208	0.827	
sizesmall	10.41166	3.82430	2.722	**
mxPH	−3.96116	2.48281	−1.595	
Cl	−0.05216	0.03168	−1.646	
NO3	−0.89423	0.35151	−2.544	*
PO4	−0.05914	0.01118	−5.291	***

- Sixteen weights become seven, built from `size`, `mxPH`, `Cl`, `NO3` and `PO4`.
- R^2 barely moves, from 0.37 to 0.35, and the typical miss stays at 17.5.
- Same accuracy, fewer weights.

These t values come after the same data chose the model, so they look better than they are. Honest selection needs a separate validation set.

What One Coefficient Claims

Take `NO3` $= -0.89$ from the smaller model.

The claim

One more unit of nitrate goes with 0.89 less `a1` on average, **with every other predictor held fixed**.

Not the claim

- **Not cause.** Cutting nitrate is not shown to raise algae.
- **Not importance.** A weight depends on its units; `NH4`'s 0.0016 is tiny because `NH4` runs into the thousands.
- **Not outside the data.** "Held fixed" means nothing for combinations never observed.

Predicting One River by Hand

A small river with `mxPH` $= 8.0$, `Cl` $= 40$, `NO3` $= 3.0$ and `PO4` $= 120$. Multiply each value by its weight, then add:

$$\begin{aligned}\hat{y} &= 57.19 + 10.41 - 3.961(8.0) - 0.0522(40) - 0.8942(3.0) - 0.0591(120) \\ &= 24.05 \approx 24.0\end{aligned}$$

`sizesmall` adds 10.41 because the river is small; a large river would add nothing. The result is the predicted average for rivers like this one, not a range for a single new sample.

Four Assumptions, Four Failures

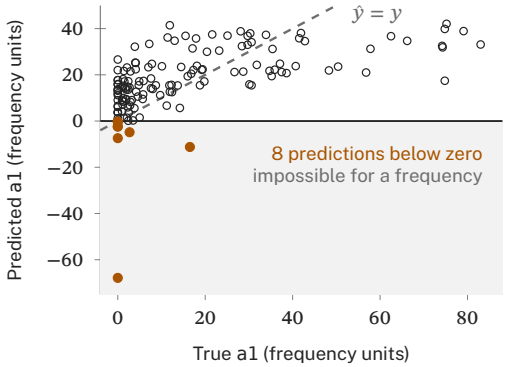
DALHOUSIE UNIVERSITY

Assumption	What it says	If it breaks	How you would notice
Linearity	The average target is a weighted sum of the predictors	Estimates are biased; the model is wrong, not merely noisy	Curvature in residuals against fitted values
Independence	One case's miss says nothing about another's	Standard errors and tests can be wrong	Time, space or group patterns in the residuals
Constant variance	Misses have the same spread for every case	Coefficients stay unbiased, but tests and intervals are wrong	A funnel shape in residuals against fitted values
Normal errors	Misses follow a bell curve around zero	t values and intervals lose calibration	A bent residual Q-Q plot

The skewed algae misses point at the last row. `plot(la1)` draws the residual plots that check all four.

Predicting New Rivers

- On 140 rivers the model never saw, the test MSE is 295.5.
- **Eight predictions are negative**, and there are no negative algae frequencies.
- Least squares fits a plane over every value; $y \geq 0$ was never part of the criterion. Cutting at zero afterwards is a repair, not a model.



Predicting New Rivers in R and Python

R

```r
test.algae <- read.csv("data/test_algae.csv")
algae.sols <- read.csv("data/algae_sols.csv")
clean.test.algae <- mixed_knn_impute(test.algae,
    k = 10, donors = algae[, 1:11])
preds <- predict(final.la1, clean.test.algae)
mean((preds - algae.sols$a1)^2)   # 295.5436
plot(algae.sols$a1, preds)
abline(0, 1, col = 'red', lty = 2)
```

Python

```python
import matplotlib.pyplot as plt
test = pd.read_csv("data/test_algae.csv")
algae_sols = pd.read_csv("data/algae_sols.csv")
# donors: the training rows, never the test rows
imp = KNNImputer(n_neighbors=10).fit(raw_numeric)
test[num] = imp.transform(test[num])
preds = final.predict(test)  # matched by name
((preds - algae_sols["a1"])**2).mean()
plt.scatter(algae_sols["a1"], preds)
plt.axline((0, 0), slope=1, color="red", ls="--")
```

Fill the test gaps from the **training** rows only, never from other test rows. Both fitted models match the new data's columns by name.

Linear discriminant analysis

A third bargain, keep the linear form, but aim it at the classes instead of at the target values.

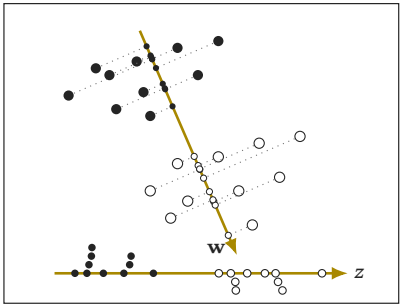
One Score per Case

The idea. Combine the predictors into a single score, a weighted sum like a regression,

$$z = w_1 X_1 + w_2 X_2 + \cdots + w_p X_p,$$

with weights chosen so the classes pull apart.

- Each case lands at one point on a line, the discriminant axis.
- A new case is classified by where its score falls on that line.
- Regression aimed its weighted sum at a numeric target; LDA aims it at the classes.



Fisher's Separation Score

For two classes, a set of weights is judged on the scores z it produces,

$$J = \frac{(\bar{z}_1 - \bar{z}_2)^2}{s_z^2} = \frac{(\text{gap between the class means})^2}{\text{spread within the classes}}$$

The top rewards separation

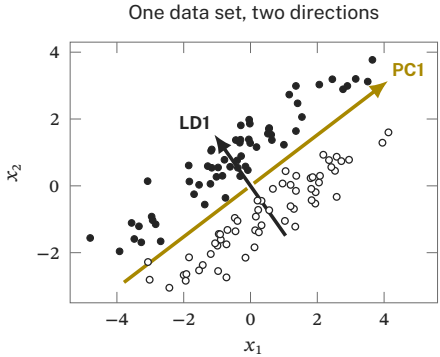
$\bar{z}_1$ and $\bar{z}_2$ are the average scores of each class. Far apart is good.

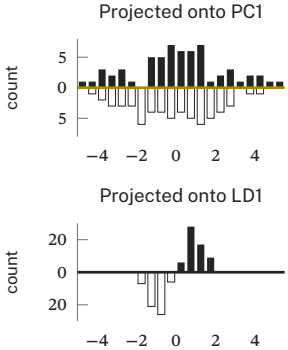
The bottom charges overlap

s_z^2 is the score variance inside each class, pooled in proportion to class size. Tight classes are good.

Squaring hides the sign of the weights; doubling every weight leaves J unchanged.

LDA Is Not PCA





Simulated data, two long classes side by side.

PCA never sees the labels. It follows the direction in which the data spread most.

LDA uses the labels. It follows the direction that separates the classes.

Here the two are **87.9° apart**. On PC1 the classes overlap; on LD1 they split cleanly, a separation of 0.14 against 5.05.

Fitting LDA on the Iris Flowers

R

```r
library(MASS)
l <- lda(Species ~ ., iris)
l$prior   # 0.3333 0.3333 0.3333
l$means   # one row per species
```

Python

```python
from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import \
     LinearDiscriminantAnalysis as LDA
X, y = load_iris(return_X_y=True, as_frame=True)
l = LDA().fit(X, y)
l.priors_   # 0.3333 0.3333 0.3333
l.means_    # one row per species
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
setosa	5.006	3.428	1.462	0.246
versicolor	5.936	2.770	4.260	1.326
virginica	6.588	2.974	5.552	2.026

- The **priors** are each species' share of the 150 `iris` flowers, a third each.
- The **class means** are what LDA pulls apart. Petal length and width already separate the species.

The Discriminant Weights

DALHOUSIE UNIVERSITY

Weights, `l$scaling`

	LD1	LD2
Sepal.Length	0.8294	0.0241
Sepal.Width	1.5345	2.1645
Petal.Length	−2.2012	−0.9319
Petal.Width	−2.8105	2.8392

Share of the separation

LD1	LD2
0.9912	0.0088

- Three species give at most two axes. **LD1 carries 99%** of the separation, so one axis is nearly enough.
- Signs are arbitrary. Flipping a whole column gives the same classifier.
- Sizes depend on each predictor's units, so they are not importances.
- The share is of *separation*, not of variance, unlike PCA.

scikit-learn's `l.scalings_` prints weights about 1% larger, from a different divisor; its predictions are the same.

Predicting With LDA

R

```r
sp <- sample(1:150, 100)
tr <- iris[sp, ]; ts <- iris[-sp, ]
l <- lda(Species ~ ., tr)
preds <- predict(l, ts)
# rows = predicted class
(mtrx <- table(preds$class, ts$Species))
1 - sum(diag(mtrx)) / sum(mtrx)
```

Python

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import (confusion_matrix,
                             accuracy_score)
tr, ts, ytr, yts = train_test_split(
    X, y, train_size=100, random_state=3141)
l = LDA().fit(tr, ytr)
preds = l.predict(ts)
mtrx = confusion_matrix(yts, preds)  # rows = TRUE
1 - accuracy_score(yts, preds)       # 0.02
```

Read the margins first. R puts the predicted class down the rows and Python puts the true class there, so the two tables are mirror images.

Without `set.seed()` or `random_state`, every run draws a different split and a different table.

Reading the Result

pred. ↓ true →	setosa	versicolor	virginica
setosa	**16**	0	0
versicolor	0	**18**	0
virginica	0	1	**15**

- One versicolor is called virginica, so the error rate is $1/50 = 0.02$.
- Every setosa is right, as the class means suggested; the hard boundary is versicolor against virginica.
- With 50 test flowers, one mistake moves the error rate by 0.02, so 0.02 and 0.04 are one flower apart.

Three Models, Three Bargains

DALHOUSIE UNIVERSITY

kNN

- Stores the training cases and predicts from the K nearest.
- Choose K by validation, starting near $\sqrt{n}$.
- Assumes no shape; needs scaling and is slow to predict.

Linear regression

- A weighted sum fitted by least squares.
- Readable weights and a `summary()` to test them.
- Assumes a straight-line shape.

LDA

- One score per case that separates the classes.
- Maximizes the gap between class means over the spread.
- Assumes Normal classes with a shared spread.

The more a model assumes, the more it explains, and the more ways it can be wrong.